Group29_Checkpoint 2

April 18, 2026

1 Data Mining 2026 — Project Notebook

Course: Data Mining
Project track: Track A: Standard Analysis Project

Group members:

- Victor Bjerggaard Thøstesen
- Nikolai Mose Sannerum
- Frederik Hansen Baattrup

Dataset:

During the project we look at game statistics for Premier league matches since 2005. The datasets from 2003/2004 and 2004/2005 seasons are disregarded due to reading issues.

Some features of the full datasets are combined into a master dataframe. This dataframe contains the features: "HomeTeam", "AwayTeam", "Full Time Result (FTR)", "Referee", "Full Time Home Goal (FTHG)", "Full Time Away Goal (FTAG)", "Home Shots (HS)", "Away Shots (AS)", "Home Shots On Target (HST)", "Away Shots On Target (AST)", "Home Fouls (HF)", "Away Fouls (AF)", "Home Corner (HC)", "Away Corner (AC)", "Home Yellow (HY)", "Away Yellow (Ay)", "Home Red (HR)", "Away Red (AR)", "Average Home Win Odds (AvgH)", "Average Draw Odds (AvgD)", "Average Away Win Odds (AvgA)", "Date" and "Season".

Initial task description (Module 1 perspective):

For the first part of the project we want to analyze our dataset using clustering. Our tentative analysis goal is to pick out a set of features from the dataset which can be meaningfully clustered and compared to a ground truth. First we apply unsupervised clustering to group football matches using in game statistics. The chosen features will be combined and transformed in order to reduce dimensions and make the matches more comparable. Outcome-related variables FTHG, FTAG, FTR, HTR and odds are excluded from clustering to prevent data leak and reserved for evaluation. After normalizing the numeric features, clustering algorithms are applied to identify match groupings. The FTR column serves as ground truth to assess alignment between clusters and actual outcomes (home win, away win, draw). Cluster evaluation metrics includes contingency tables and cluster purity measurements.

1.1 0. Reproducibility and Setup

[1]:
```
import sys
print(sys.version)

import numpy as np
import pandas as pd
import os
import math
import matplotlib.pyplot as plt
import networkx as nx
from matplotlib.patches import Patch
from tqdm import tqdm
from sklearn.cluster import DBSCAN, KMeans
from sklearn.manifold import TSNE
from scipy.spatial.distance import cdist


RANDOM_SEED = 42
np.random.seed(RANDOM_SEED)
```

```
3.13.7 (v3.13.7:bcee1c32211, Aug 14 2025, 19:10:51) [Clang 16.0.0
(clang-1600.0.26.6)]
```

1.2 1. Dataset Description and Loading

1.2.1 Dataset Overview

- Source: https://www.football-data.co.uk/englandm.php?fbclid=IwY2xjawQVBq9leHRuA2FlbQIxMABicml QTbB8sBlRDK8vETlRUAA3XgndZg_aem_OCN-0ddvAe0Fob_1GlxMdg
- Number of instances (matches): 7860
- Number of features / entities: 23
- Missing values: No missing values after preprocessing. However when the seasons were combined we used only the average odds (provided by different websites for different seasons).

[2]:
```
# --- Data Extraction Function ---
# Here we define a function to extract the chosen relevant data features from␣
↪the raw CSV files.
# This function will look for the average odds columns and handle cases where␣
↪they might be missing.

def extract_data(df, filename):
    """
    Extracts average in game features and home/draw/away odds from CSV file.
    Prefers BbAvH/D/A, otherwise falls back to AvgH/D/A.
    Adds season column extracted from filename.
```

```
    parameters:
    df: DataFrame containing the raw data from the CSV file.
    filename: The name of the CSV file, used to extract the season information.

    return: A DataFrame with the extracted features and season information, or␣
↪None if required columns are missing.

    """

    primary_cols = ['HomeTeam', 'AwayTeam', 'FTR', 'Referee', 'FTHG',
                    'FTAG', 'HS', 'AS', 'HST', 'AST', 'HF', 'AF',
                  'HC', 'AC', 'HY', 'AY', 'HR', 'AR', "BbAvH", "BbAvD", "BbAvA",␣
↪'Date']
    fallback_cols = ['HomeTeam', 'AwayTeam', 'FTR', 'Referee', 'FTHG',
                    'FTAG', 'HS', 'AS', 'HST', 'AST', 'HF', 'AF',
                  'HC', 'AC', 'HY', 'AY', 'HR', 'AR', "AvgH", "AvgD", "AvgA",␣
↪'Date']

    season = os.path.splitext(filename)[0]

    # Trying to find Betbrain columns first
    if all(col in df.columns for col in primary_cols):
        result = df[primary_cols].copy()
        result.columns = ['HomeTeam', 'AwayTeam', 'FTR', 'Referee', 'FTHG',
                          'FTAG', 'HS', 'AS', 'HST', 'AST', 'HF', 'AF',
                          'HC', 'AC', 'HY', 'AY', 'HR', 'AR', "AvgH", "AvgD",␣
↪"AvgA", 'Date']
        print(f'Found primary columns: BbAvH, BbAvD, BbAvA for {filename}')

    # Else trying to find fallback columns
    elif all(col in df.columns for col in fallback_cols):
        result = df[fallback_cols].copy()
        print(f'Using fallback columns: AvgH, AvgD, AvgA for {filename}')

    # If neither set of columns is found, print a warning and return None
    else:
        print(f"  Warning: Required odds columns not found in {filename}")
        return None

    # Add season column
    result["Season"] = season

    return result
```

[3]:
```
# --- User settings begin ---
dataPath = "Data" # Define the folder where the raw CSV files are located.
# --- User settings end ---
```

```python
new_data = []

for file in os.listdir(dataPath):
    if file.endswith(".csv"):
        path = os.path.join(dataPath, file)
        try:
            df = pd.read_csv(path)
        except Exception as e:
            print(f"  Error reading {file}: {e}")
            continue

        extracted = extract_data(df, file)

        if extracted is not None:
            new_data.append(extracted)

# Combine everything into one dataframe
final_df = pd.concat(new_data, ignore_index=True)
```

```
Found primary columns: BbAvH, BbAvD, BbAvA for 20072008premierleague.csv
Found primary columns: BbAvH, BbAvD, BbAvA for 20112012premierleague.csv
Found primary columns: BbAvH, BbAvD, BbAvA for 20132014premierleague.csv
Using fallback columns: AvgH, AvgD, AvgA for 20242025premierleague.csv
  Error reading 20042005premierleague.csv: 'utf-8' codec can't decode byte 0xa0
in position 75614: invalid start byte
Using fallback columns: AvgH, AvgD, AvgA for 20202021premierleague.csv
Using fallback columns: AvgH, AvgD, AvgA for 20192020premierleague.csv
Found primary columns: BbAvH, BbAvD, BbAvA for 20082009premierleague.csv
Found primary columns: BbAvH, BbAvD, BbAvA for 20062007premierleague.csv
Found primary columns: BbAvH, BbAvD, BbAvA for 20152016premierleague.csv
Using fallback columns: AvgH, AvgD, AvgA for 20222023premierleague.csv
Found primary columns: BbAvH, BbAvD, BbAvA for 20102011premierleague.csv
Found primary columns: BbAvH, BbAvD, BbAvA for 20092010premierleague.csv
Found primary columns: BbAvH, BbAvD, BbAvA for 20122013premierleague.csv
Using fallback columns: AvgH, AvgD, AvgA for 20252026premierleague.csv
  Error reading 20032004premierleague.csv: Error tokenizing data. C error:
Expected 57 fields in line 305, saw 72

Found primary columns: BbAvH, BbAvD, BbAvA for 20172018premierleague.csv
Found primary columns: BbAvH, BbAvD, BbAvA for 20162017premierleague.csv
Found primary columns: BbAvH, BbAvD, BbAvA for 20052006premierleague.csv
Using fallback columns: AvgH, AvgD, AvgA for 20232024premierleague.csv
Using fallback columns: AvgH, AvgD, AvgA for 20212022premierleague.csv
Found primary columns: BbAvH, BbAvD, BbAvA for 20182019premierleague.csv
Found primary columns: BbAvH, BbAvD, BbAvA for 20142015premierleague.csv
```

[4]:
```
print(f"\n Final dataset:")
final_df = final_df.dropna(subset=["HomeTeam", "AwayTeam"]) # 2014/2015 season␣
↪has a row of NaN that is removed here
final_df
```

```
 Final dataset:
```

[4]:
```
           HomeTeam     AwayTeam FTR        Referee  FTHG  FTAG    HS    AS  \
0       Aston Villa    Liverpool   A        M Riley   1.0   2.0  10.0  17.0
1            Bolton    Newcastle   A          C Foy   1.0   3.0  13.0   7.0
2             Derby   Portsmouth   D         M Dean   2.0   2.0  12.0  12.0
3           Everton        Wigan   H  M Clattenburg   2.0   1.0  12.0  14.0
4     Middlesbrough    Blackburn   A     A Marriner   1.0   2.0  10.0   4.0
…               …            …  ..             …   …   …   …   …
7855           Hull   Man United   D      L Probert   0.0   0.0  16.0   7.0
7856      Leicester          QPR   H       M Oliver   5.0   1.0  22.0  18.0
7857       Man City  Southampton   H          C Foy   2.0   0.0  15.0  13.0
7858      Newcastle     West Ham   H     M Atkinson   2.0   0.0  17.0   4.0
7859          Stoke    Liverpool   H       A Taylor   6.0   1.0  15.0  13.0

      HST  AST  …   AC   HY   AY   HR   AR  AvgH  AvgD  AvgA      Date  \
0     6.0  7.0  …  2.0  4.0  2.0  0.0  0.0  3.67  3.23  1.95  11/08/07
1     9.0  5.0  …  3.0  1.0  1.0  0.0  0.0  2.41  3.18  2.80  11/08/07
2     5.0  6.0  …  6.0  1.0  2.0  0.0  0.0  2.85  3.19  2.38  11/08/07
3     8.0  4.0  …  2.0  0.0  0.0  0.0  0.0  1.66  3.45  5.25  11/08/07
4     6.0  4.0  …  3.0  3.0  4.0  0.0  0.0  2.41  3.18  2.81  11/08/07
…   …  …  …  …  …  …  …  …  …   …   …   …        …
7855  6.0  1.0  …  1.0  2.0  2.0  0.0  1.0  3.14  3.55  2.21  24/05/15
7856  7.0  2.0  …  6.0  0.0  0.0  0.0  0.0  1.56  4.21  5.71  24/05/15
7857  6.0  4.0  …  4.0  1.0  1.0  0.0  0.0  1.56  4.27  5.61  24/05/15
7858  4.0  1.0  …  3.0  2.0  1.0  0.0  0.0  1.77  3.84  4.42  24/05/15
7859  9.0  4.0  …  9.0  4.0  2.0  0.0  0.0  3.42  3.48  2.12  24/05/15

                     Season
0     20072008premierleague
1     20072008premierleague
2     20072008premierleague
3     20072008premierleague
4     20072008premierleague
…                       …
7855  20142015premierleague
7856  20142015premierleague
7857  20142015premierleague
7858  20142015premierleague
7859  20142015premierleague
```

```
[7860 rows x 23 columns]
```

[5]:
```python
def encode_results(results):
    """
    Endcoding of results
    1: HOME WIN, -1: AWAY WIN, 0: DRAW

    parameters:
    results (list of dicts): results to encode

    returns:
    encoding (numpy array): encoded results
    """
    encoding = np.zeros(len(results)) # Initialize encoding array

    for i in range(len(results)):
        if results[i] == "H":
            encoding[i] = 1 # Home win
        elif results[i] == "A":
            encoding[i] = -1 # Away win
        else:
            encoding[i] = 0 # Draw
    return encoding
data = final_df.copy()
ground_truth = encode_results(data['FTR'])
print("Amount of home wins: ", np.sum(ground_truth == 1))
print("Amount of away wins: ", np.sum(ground_truth == -1))
print("Amount of draws: ", np.sum(ground_truth == 0))
```

```
Amount of home wins:  3586
Amount of away wins:  2368
Amount of draws:  1906
```

1.3 Clustering (Part I)

1.3.1 Exploratory data analysis

New Features and Normalization To make matches more comparable, new features were engineered as the difference between home and away statistics. A positive value indicates that the home team has a higher value for the given metric.

The following features were constructed:

- Shot difference ($HS - AS$)
- Shots on target difference ($HST - AST$)
- Foul difference ($HF - AF$)
- Corner difference ($HC - AC$)
- Difference in average betting odds ($AvgH - AvgA$)

After feature engineering, the variables were **normalized using z-score normalization** to ensure

comparable scales across features.

Data insights: Finally, histograms of the normalized features were plotted to inspect their distributions. The following insights are observed: - All variables appear roughly symmetric and centered around zero, which is expected after z-score normalization. This indicates that, on average, neither the home nor away team consistently dominates these metrics across matches.

- Most feature distributions appear approximately Gaussian after normalization. The exceptions are RC_diff and YC_diff, which are highly discrete due to the low and integer-valued nature of card events. The largely unimodal and continuous shape of the other features suggests that the data may not naturally separate into distinct groups across all variables. Any potential clustering structure may therefore rely more heavily on the discrete features.

Overall, this indicates that clusters, if present, are likely to be weakly separated, which may make clustering more challenging.

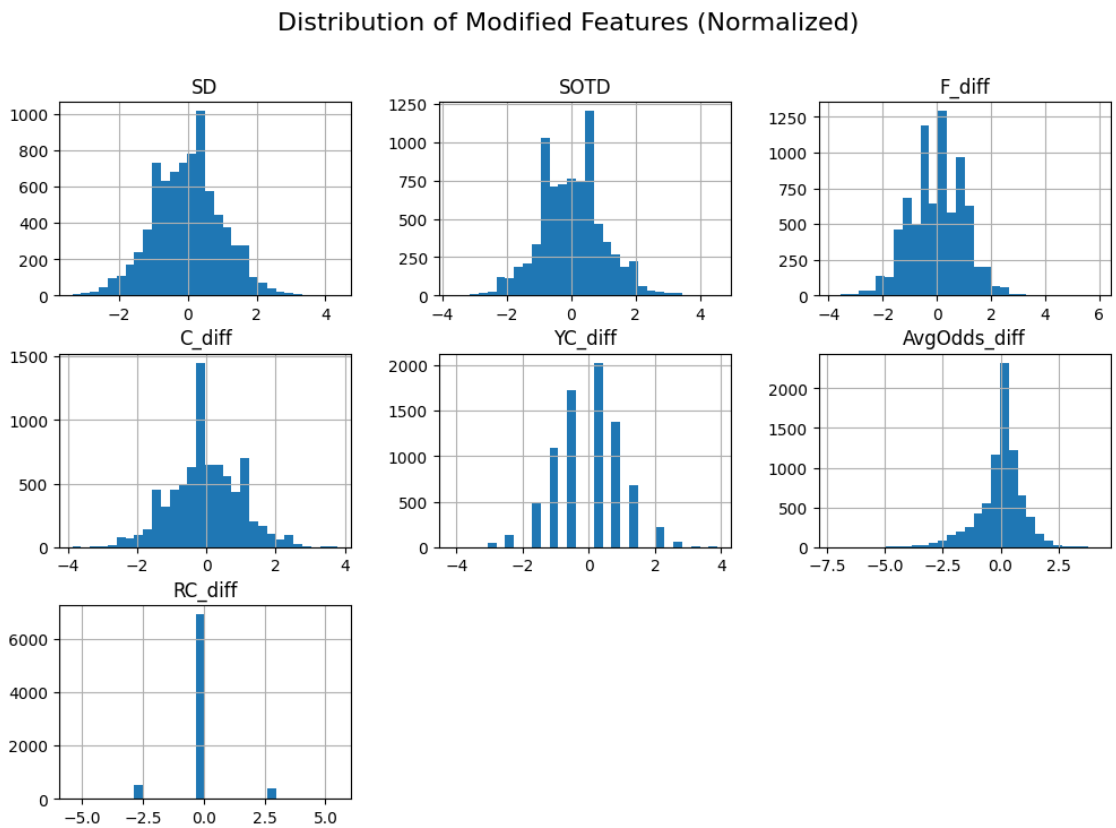
Scatter Plots: The scatter plots below show the relationships between the normalized features and the ground truth, Full Time Result (FTR), which indicates whether the home team won (1), drew (0), or lost (-1). Each scatter plot illustrates how various match statistics, such as shot differences, foul differences, and card differences, are distributed against FTR:

- **SD vs FTR**: Shot difference vs Full Time Result
- **SOTD vs FTR**: Shots on target difference vs Full Time Result
- **F_diff vs FTR**: Foul difference vs Full Time Result
- **C_diff vs FTR**: Corner difference vs Full Time Result
- **AvgOdds_diff vs FTR**: Difference in average betting odds vs Full Time Result

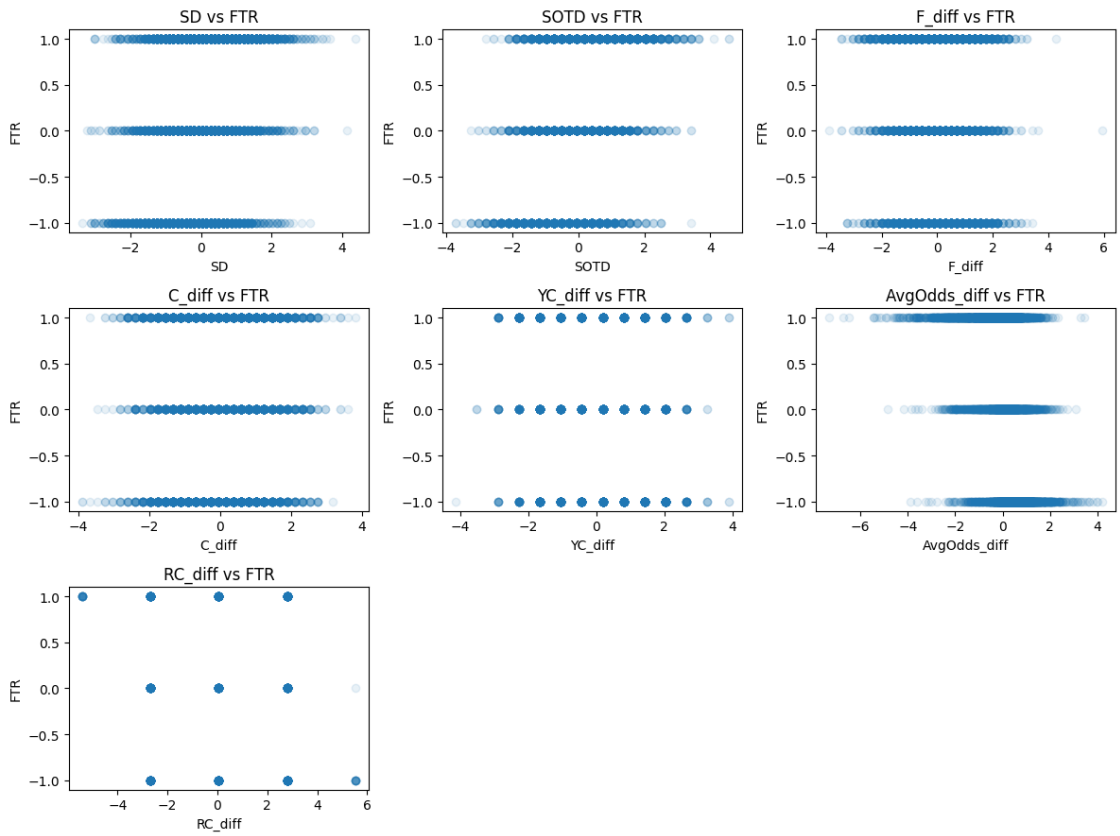
Each scatter plot demonstrates the relationship between these engineered features and FTR (ground truth). This was done to get an idea of which features that seems important when comparing to ground truth. It seems that there are some correlation between some of the features and ground truth, though they still might be hard to cluster.

[6]:
```
# --- Creating new features by calculating the difference between home and away␣
↪statistics (Positive means home team have more) ---
modified_stats = pd.DataFrame({'SD': data['HS'] - data['AS'],
                               'SOTD': data['HST'] - data['AST'],
                               'F_diff': data['HF'] - data['AF'],
                               'C_diff': data['HC'] - data['AC'],
                               'YC_diff': data['HY'] - data['AY'],
                               'AvgOdds_diff': data['AvgH'] - data['AvgA'],
                               'RC_diff': data['HR'] - data['AR']
                               })

# --- Normalize the new features with average and standard deviation (Z-score␣
↪normalization) ---
modified_stats = (modified_stats - modified_stats.mean()) / modified_stats.std()

# -- Plot as histogram to see the distribution of the new features ---
modified_stats.hist(bins=30, figsize=(12, 8))
```

```
plt.suptitle('Distribution of Modified Features (Normalized)', fontsize=16)
plt.show()
```



[7]:
```
n_cols = 3  # number of columns in the subplot grid
n_features = len(modified_stats.columns)
n_rows = math.ceil(n_features / n_cols)

plt.figure(figsize=(4 * n_cols, 3 * n_rows))

for i, column in enumerate(modified_stats.columns):
    plt.subplot(n_rows, n_cols, i + 1)
    plt.scatter(modified_stats[column], ground_truth, alpha=0.1)
    plt.title(f'{column} vs FTR')
    plt.xlabel(column)
    plt.ylabel('FTR')

plt.tight_layout()
plt.show()
```



1.4 DBSCAN

Tuning the parameter ϵ Before running DBSCAN on the dataset, we try to find the best hyperparameter for ϵ.

First we want to find the "border object" or "first valley" (non differentiable point). This is done by defining a function that for each point find the distance to the k-nearest neighbor. This distance is the distance to the MinPts-nearest nighbor where MinPts is set to $2d - 1$ where d is the number of features (Slide 22 lecture 3).

The distance measure used, is both the eucledean distance and manhattan distance.

Afterwards we plot the distances in descending order.

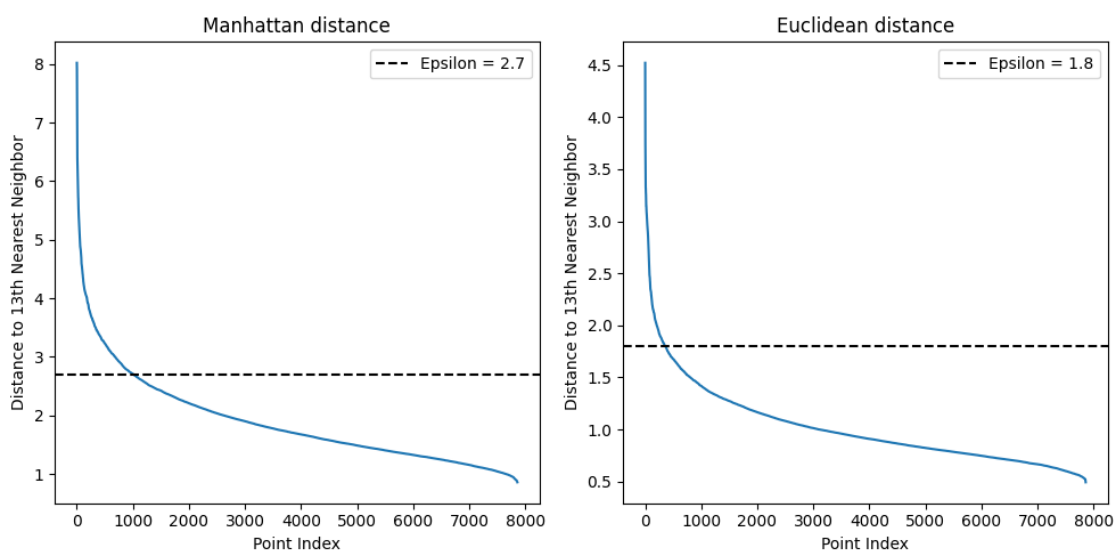
[8]:
```
# -- Calculating K nearest neighbours --
def find_k_nearest_neighbor_dist(data, dist: str = 'euclidean', minpts=None,␣
 ↪use_tqdm=True):
    """
    Compute the distance to the k-th nearest neighbor for each point.

    Parameters:
        data (numpy array): Shape (n_samples, d_features)
```

```
        dist (str): The distance metric to use (default 'euclidean')
        minpts (int): Number of neighbors to consider (default 2*d - 1 -- See␣
↪slide 22 lecture 3)
        use_tqdm (bool): Show progress bar for large datasets
        minpts (int): Number of neighbors to consider (default 2*d - 1 -- See␣
↪slide 22 lecture 3)
        use_tqdm (bool): Show progress bar for large datasets

    Returns:
        distances_sorted (numpy array): k-th nearest neighbor distances sorted␣
↪descending
        minpts (int): The k used
    """
    n, d = data.shape
    if minpts is None:
        minpts = 2 * d - 1

    # Compute full pairwise distance matrix (n x n)
    pairwise_dist = cdist(data, data, metric=dist) # Manhattan distance is used

    # Set diagonal to np.inf to ignore self-distance
    np.fill_diagonal(pairwise_dist, np.inf)

    distances = np.zeros(n)

    iterator = tqdm(range(n), disable=not use_tqdm)
    for i in iterator:
        # Sort distances for this point
        sorted_dists = np.partition(pairwise_dist[i], minpts)  # faster than␣
↪full sort
        distances[i] = sorted_dists[minpts]  # minpts-th nearest neighbor

    # Return distances sorted descending
    return np.sort(distances)[::-1], minpts



y_man, minpts_man = find_k_nearest_neighbor_dist(modified_stats.values,␣
↪dist='cityblock')
print('Found value for minpts (Manhattan):', minpts_man)

y_euclidean, minpts_euclidean = find_k_nearest_neighbor_dist(modified_stats.
↪values, dist='euclidean')
print('Found value for minpts (Euclidean):', minpts_euclidean)
```

```
100%|      | 7860/7860 [00:00<00:00, 16958.29it/s]

Found value for minpts (Manhattan): 13
```

```
100%|      | 7860/7860 [00:00<00:00, 17686.13it/s]
Found value for minpts (Euclidean): 13
```

[9]:
```
# -- Plotting K nearest Neighbor distances---
x_man = np.arange(len(y_man))
x_euclidean = np.arange(len(y_euclidean))
epsilon_man = 2.7
epsilon_euclidean = 1.8
fig, ax = plt.subplots(1,2, figsize=(10, 5))
ax[0].plot(x_man, y_man)
ax[0].set_xlabel("Point Index")
ax[0].set_ylabel(f"Distance to {minpts_man}th Nearest Neighbor")
ax[0].set_title(f"Manhattan distance")
ax[0].axhline(epsilon_man, c='k', ls='--', label=f'Epsilon = {epsilon_man}')
ax[0].legend()

ax[1].plot(x_euclidean, y_euclidean)
ax[1].set_xlabel("Point Index")
ax[1].set_ylabel(f"Distance to {minpts_euclidean}th Nearest Neighbor")
ax[1].set_title(f"Euclidean distance")
ax[1].axhline(epsilon_euclidean, c='k', ls='--', label=f'Epsilon =␣
 ↪{epsilon_euclidean}')
ax[1].legend()
plt.tight_layout()
```



As seen above, it is hard to to determine the right ϵ as there is no clear valley in the plot in any of the plots. This is a sign that there isn't some clearly defined clusters in the data, as the density

acts continously rather than discretely. However we choose $\epsilon = 1.8$ for the euclidean distance and $\epsilon = 2.7$ for the manhattan distance, and continue to DBSCAN.

DBSCAN clustering

[10]:
```
dbscan = DBSCAN(eps=epsilon_euclidean, min_samples=minpts_euclidean)
dbscan.fit(modified_stats.values)
labels = dbscan.labels_
print("DBSCAN Cluster Labels:", np.unique(labels)) # -1 corresponds to noise␣
↪points, other integers correspond to cluster labels
```

```
DBSCAN Cluster Labels: [-1  0  1  2]
```

It is observed that the result of DBSCAN when using the manhattan distance is exstremely sensitive to the choice of epsilon. Choosing: - $\epsilon = 2.8$ results in 1 cluster and some noise - $\epsilon = 2.75$ results in 2 clusters and some noise. - $\epsilon = 2.7$ results in 4 clusters and some noise.

This is a sign that the data is ill suited for clustering using the manhattan distance. When using the euclidean distance, the result is more stable.

There is no clear valley in the plot of the k-nearest neighbor distances, so the clusters obtained may not be well defined. As seen above we obtain three clusters using the euclidean distance (0, 1, 2) and some noise (-1).

To evaluate the clustering, we define two functions, one makes the contingency table based on the FTR ground truth, and the other evaluates the purity score. - The contingency table tells us how many points of each ground truth the clusters contain. - The purity measure tells us how many points in the cluster belong to the same (most prominent) ground truth.

[11]:
```
def contingency_table(C, T, ignore_noise=True):

    """
    Parameters:
        C(numpy array):       Clusters obtained by a clustering algorithm as a␣
↪nx1 vector
        T(numpy array):       Ground-truth cluster assignments as a nx1 vector

    Returns:
        ctable:   a num_clusters_of_C x num_cluster_of_T matrix containing the␣
↪overlaps among the different clusters
    """

    if ignore_noise:
        mask = (C != -1)
        C_masked = C[mask]
        indices_masked = np.where(mask)[0]
    else:
        C_masked = C
        indices_masked = np.arange(len(C))
```

```python
    clusters = np.unique(C_masked)
    classes  = np.unique(T)   # include all classes, not just masked

    ctable = np.zeros((len(clusters), len(classes)), dtype=int)

    for i, c in enumerate(clusters):
        for j, t in enumerate(classes):
            # count overlap only for points that are not noise
            count = np.sum((C_masked == c) & (T[indices_masked] == t))
            ctable[i, j] = count

    return ctable



def purity_score(ctable):
    """
    Parameters:
        ctable (numpy array):  Contingency table of shape (num_clusters,␣
 ↪num_classes) containing the overlaps among the different clusters

    Returns:
        purity(float):   a float, the purity of clustering result
    """

    clusters, gt_classes = ctable.shape
    purity = np.zeros(clusters)

    for i in range(clusters):
        purity[i] = np.max(ctable[i])/np.sum(ctable[i]) if np.sum(ctable[i]) >␣
 ↪0 else 0
    return purity



# --- Contingency table to compare the cluster labels with the ground truth ---
c_table = contingency_table(labels, ground_truth)
print("Contingency Table:\n", c_table)

# --- Calculate the purity of the clustering ---
purity = purity_score(c_table)
print(f"Purity score of the clustering: {purity}")
```

```
Contingency Table:
 [[2083 1671 3134]
 [  73  127  322]
 [ 185   94   83]]
```

```
Purity score of the clustering: [0.45499419 0.61685824 0.51104972]
```

First, looking at the contingency table, the rows correspond to different clusters, while the columns correspond to ground truth (1: Homewin, 2: draw, 3: Homeloose). Since each row in the contingency table is heavily mixed, the clustering seems to not obtain the ground truth. This can also be seen in the purity, which calculates the proportion of elements in the cluster that belong to the most frequent class. All purities is between 45 and 64 percent, meaning no cluster is pure compared to the ground truth.

If one cluster should be highlighted it would be cluster 1. It is smaller but dominated by home losses, giving it the highest purity. This suggests DBSCAN found a small region where matches with similar characteristics tend to result in home losses.

1.5 K-Means++ clustering

Since we are comparing to a known ground truth with 3 outcomes (Home Win, Draw, Away Win) we run K-means++ clustering with $k = 3$ clusters. For the same reason we evaluate the clustering using two external measures: Contingency table and purity.

[12]:
```
k_means = KMeans(n_clusters=3).fit(modified_stats.values)
labels_kmeans = k_means.labels_

# --- making a contengincy tabele to compare the cluster labels with the ground␣
↪truth ---
c_table = contingency_table(labels_kmeans, ground_truth)
print("Contingency Table:\n", c_table)

# --- Calculate the purity of the clustering ---
purity = purity_score(c_table)
print(f"Purity score of the clustering: {purity}")
```

```
Contingency Table:
 [[ 774  635 1042]
 [ 382  576 1763]
 [1212  695  781]]
Purity score of the clustering: [0.4251326  0.64792356 0.45089286]
```

The clustering obtained by using K-Means is by no means better that the one obtained from using DBSCAN

1.5.1 Evaluation of clustering results

[13]:
```
from sklearn.metrics import adjusted_rand_score, normalized_mutual_info_score,␣
↪accuracy_score
clustering_result = {
    'DBSCAN': dbscan.labels_,
    'K-means++': k_means.labels_
}
for name, y_pred in clustering_result.items():
    print(f"\n{name}:")
```

```
    ari = adjusted_rand_score(ground_truth, y_pred)
    nmi = normalized_mutual_info_score(ground_truth, y_pred)
    print(f"ARI: {ari:.4f}, NMI: {nmi:.4f}")
```

```
DBSCAN:
ARI: 0.0006, NMI: 0.0149

K-means++:
ARI: 0.0608, NMI: 0.0511
```

1.5.2 Clustering Evaluation Results

We evaluate the clustering performance of **DBSCAN** and **K-means++** using:

- Adjusted Rand Index (ARI)
- Normalized Mutual Information (NMI)

Both clustering methods perform poorly. The ARI scores are close to zero, indicating performance no better than random assignment. Similarly, the NMI values are near zero, showing almost no agreement between predicted clusters and ground truth labels.

K-means++ performs slightly better than DBSCAN, but the improvement is marginal and still indicates weak clustering performance.

Overall, neither method successfully captures the underlying structure of the data. This suggests that:

- The data may not have a clear cluster structure, or
- Further preprocessing and parameter tuning are required.

1.6 T-SNE

To investigate whether our features could reveal meaningful clusters, we applied **t-distributed Stochastic Neighbor Embedding (T-SNE)**.

T-SNE is basically a dimensionality reduction algorithm that is useful for visualizing high-dimensional data. It works by modeling pairwise similarities between points in the high-dimensional space and trying to preserve these similarities in a lower-dimensional space. Essentially, points that are close in the original feature space should remain close in the lower dimension visualization, while distant points stay far apart.

The goal of using T-SNE here was to see if our features could form clusters and investigate properties of these clusters. This analysis serves as a useful check.

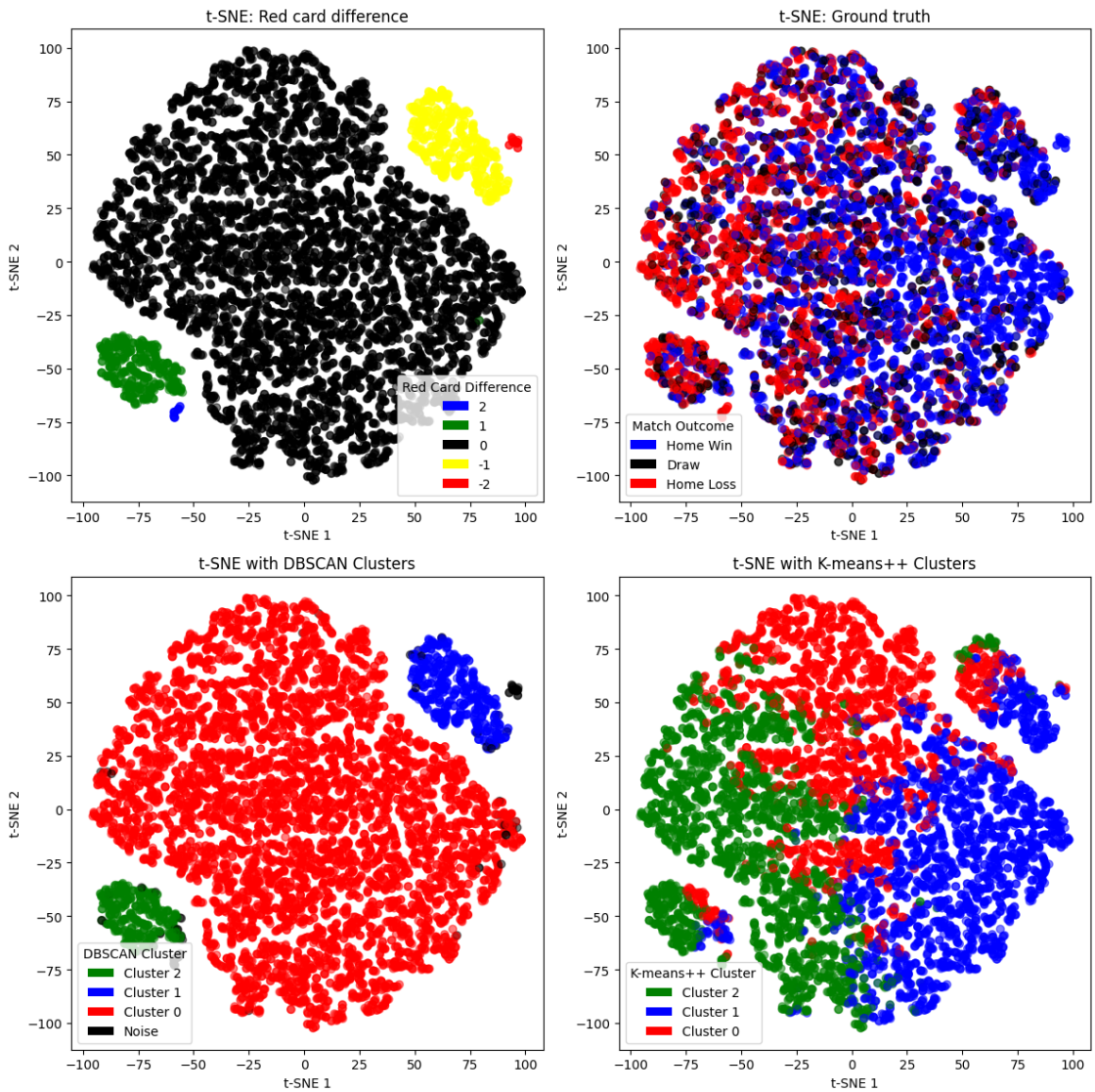
In the first cell below, T-SNE algorithm is applied to the modified stats. In the second cell, we plot the data in a two dimensional T-SNE embedding.

[14]:
```
# Fit + transform
tsne = TSNE(2, perplexity=10)
```

```
embedding = tsne.fit_transform(modified_stats.values)
```

[15]:
```
# --- Prepare evaluation values ---
eval_dataframe = pd.DataFrame({'RC_diff': data['HR'] - data['AR']})
evaluating_measure = eval_dataframe.values.flatten()
labels_dbscan = dbscan.labels_
labels_kmeans = k_means.labels_


# --- Color maps ---
color_map1 = {2: 'blue', 1: 'green', 0: 'black', -1: 'yellow', -2: 'red'}
color_map2 = {1: 'blue', 0: 'black', -1: 'red'}
color_map3 = {2: 'green', 1: 'blue', 0: 'red', -1: 'black'}
color_map4 = {2: 'green', 1: 'blue', 0: 'red'}

# --- Prepare colors ---
colors1 = [color_map1[int(val)] for val in evaluating_measure]
colors2 = [color_map2[int(val)] for val in ground_truth]
colors3 = [color_map3[int(val)] for val in labels]
colors4 = [color_map4[int(val)] for val in labels_kmeans]

# --- Legends ---
legend_1 = {2: '2', 1: '1', 0: '0', -1: '-1', -2: '-2'}
legend_2 = {1: 'Home Win', 0: 'Draw', -1: 'Home Loss'}
legend_3 = {2: 'Cluster 2', 1: 'Cluster 1', 0: 'Cluster 0', -1: 'Noise'}
legend_4 = {2: 'Cluster 2', 1: 'Cluster 1', 0: 'Cluster 0'}

# --- Create figure ---
fig, ax = plt.subplots(2, 2, figsize=(12, 12))
ax = ax.flatten()

# --- Subplot 1: Red card difference ---
ax[0].scatter(embedding[:, 0], embedding[:, 1], color=colors1, alpha=0.5)
ax[0].set_xlabel("t-SNE 1")
ax[0].set_ylabel("t-SNE 2")
ax[0].set_title("t-SNE: Red card difference")
legend_elements1 = [Patch(facecolor=color_map1[key], label=legend_1[key]) for␣
 ↪key in legend_1]
ax[0].legend(handles=legend_elements1, title="Red Card Difference")

# --- Subplot 2: Match outcome ---
ax[1].scatter(embedding[:, 0], embedding[:, 1], color=colors2, alpha=0.5)
ax[1].set_xlabel("t-SNE 1")
ax[1].set_ylabel("t-SNE 2")
ax[1].set_title("t-SNE: Ground truth")
legend_elements2 = [Patch(facecolor=color_map2[key], label=legend_2[key]) for␣
 ↪key in legend_2]
```

```
ax[1].legend(handles=legend_elements2, title="Match Outcome")

# --- Subplot 3: DBSCAN clusters ---
ax[2].scatter(embedding[:, 0], embedding[:, 1], color=colors3, alpha=0.5)
ax[2].set_xlabel("t-SNE 1")
ax[2].set_ylabel("t-SNE 2")
ax[2].set_title("t-SNE with DBSCAN Clusters")
legend_elements3 = [Patch(facecolor=color_map3[key], label=legend_3[key]) for␣
 ↪key in legend_3]
ax[2].legend(handles=legend_elements3, title="DBSCAN Cluster")

# --- Subplot 4: K-means++ clusters ---
ax[3].scatter(embedding[:, 0], embedding[:, 1], color=colors4, alpha=0.5)
ax[3].set_xlabel("t-SNE 1")
ax[3].set_ylabel("t-SNE 2")
ax[3].set_title("t-SNE with K-means++ Clusters")
legend_elements4 = [Patch(facecolor=color_map4[key], label=legend_4[key]) for␣
 ↪key in legend_4]
ax[3].legend(handles=legend_elements4, title="K-means++ Cluster")

plt.tight_layout()
plt.show()
```



Looking at the findings from T-SNE it seems that the DBSCAN algorithm has only clustered by red card diffrence in matches. This can be verified by calculating a contigency table with red card diffrence as the evaluating measure.

However, one might think that K-means++ in some way were able to find the right gradient seen in the T-SNE with ground truth, i.e. that home wins are to the right while home loss is to the left. However when looking at the generally mixed contingency table between the ground truth and the K-means++, this is not the case.

These contingency tables can be found below, and we conclude that both algorithms were not able to reproduce the ground truth.

[16]:
```
c_table1 = contingency_table(labels_dbscan, evaluating_measure)
c_table2 = contingency_table(labels_kmeans, ground_truth)
```

```
print("Contingency Table for DBSCAN with Red Card Difference:\n", c_table1)
print("Contingency Table for K-means++ with Ground Truth (Match Results):\n",␣
↪c_table2)
```

```
Contingency Table for DBSCAN with Red Card Difference:
 [[   0    0 6888    0    0]
 [   0  522    0    0    0]
 [   0    0    0  362    0]]
Contingency Table for K-means++ with Ground Truth (Match Results):
 [[ 774  635 1042]
 [ 382  576 1763]
 [1212  695  781]]
```

1.7 Disregarding disciplinary sanctions

Therefore we decided to disregard the yellow and red card features to see if that made it possible to find useful clusters. Below, the same analysis is made again.

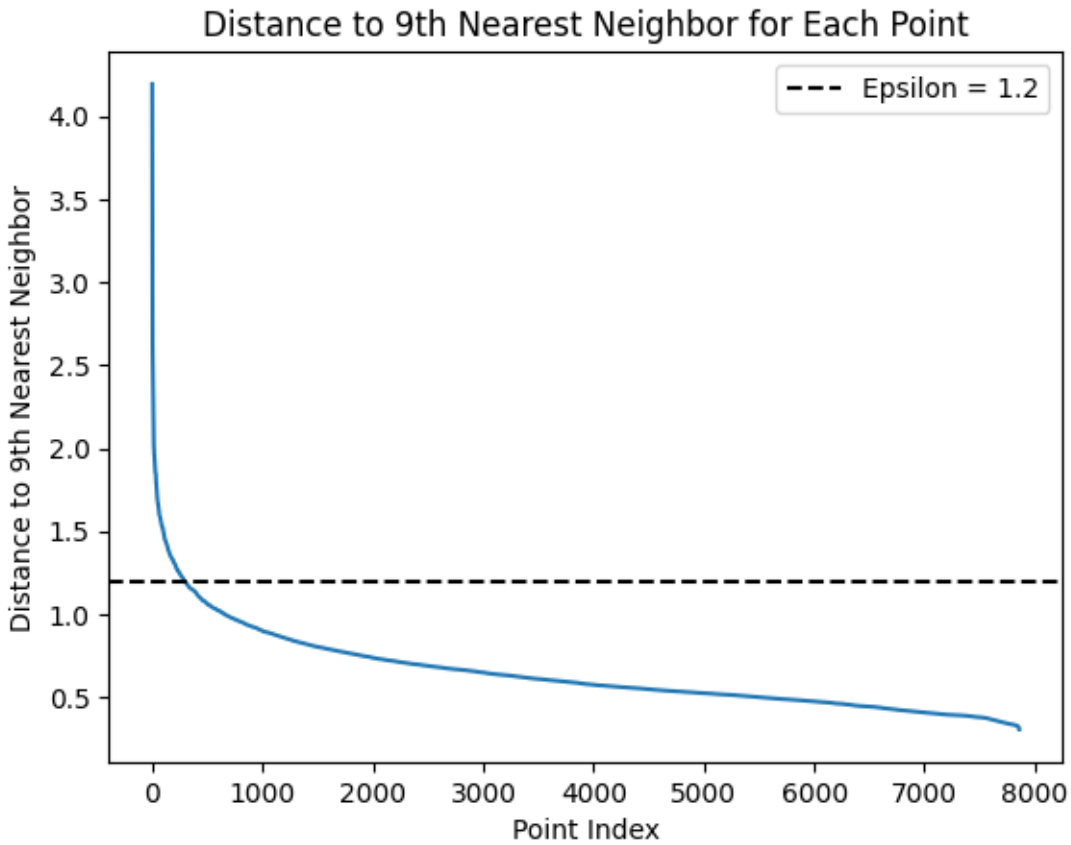
[17]:
```
# --- Creating new features again but disregarding disciplinary actions ---
modified_stats = pd.DataFrame({'SD': data['HS'] - data['AS'],
                               'SOTD': data['HST'] - data['AST'],
                               'F_diff': data['HF'] - data['AF'],
                               'C_diff': data['HC'] - data['AC'],
                               'AvgOdds_diff': data['AvgH'] - data['AvgA']
                               })

# --- Normalize the new features with average and standard deviation (Z-score␣
↪normalization) ---
modified_stats = (modified_stats - modified_stats.mean()) / modified_stats.std()
```

[18]:
```
y, minpts = find_k_nearest_neighbor_dist(modified_stats.values)
print('Found value for minpts:', minpts)
```

```
100%|      | 7860/7860 [00:00<00:00, 16067.40it/s]

Found value for minpts: 9
```

[19]:
```
# -- Plotting K nearest Neighbor distances---
x = np.arange(len(y))
epsilon = 1.2
fig, ax = plt.subplots()
ax.plot(x, y)
ax.set_xlabel("Point Index")
ax.set_ylabel(f"Distance to {minpts}th Nearest Neighbor")
ax.set_title(f"Distance to {minpts}th Nearest Neighbor for Each Point")
ax.axhline(epsilon, c='k', ls='--', label=f'Epsilon = {epsilon}')
plt.legend()
```

[19]: <matplotlib.legend.Legend at 0x14dcd7390>



Note that ϵ is updated to 1.2.

1.8 DBSCAN without displinary actions

[20]:
```
dbscan = DBSCAN(eps=epsilon, min_samples=minpts)
dbscan.fit(modified_stats.values)
labels = dbscan.labels_
print("DBSCAN Cluster Labels:", np.unique(labels)) # -1 corresponds to noise␣
↪points, other integers correspond to cluster labels
# --- Contingency table to compare the cluster labels with the ground truth ---
c_table = contingency_table(labels, ground_truth)
print("Contingency Table:\n", c_table)

# --- Calculate the purity of the clustering ---
purity = purity_score(c_table)
print(f"Purity score of the clustering: {purity}")
```

```
DBSCAN Cluster Labels: [-1  0]
Contingency Table:
```

```
 [[2357 1895 3539]]
Purity score of the clustering: [0.45424207]
```

Without the red card and yellow card difference features DBSCAN finds 1 single cluster.

1.9 K-Means without displinary actions

[21]:
```
k_means = KMeans(n_clusters=3).fit(modified_stats.values)
labels_kmeans = k_means.labels_
centroids = k_means.cluster_centers_
# --- making a contengincy tabele to compare the cluster labels with the ground␣
↪truth ---
c_table = contingency_table(labels_kmeans, ground_truth)
print("Contingency Table:\n", c_table)
# --- Calculate the purity of the clustering ---
purity = purity_score(c_table)
print(f"Purity score of the clustering: {purity}")
```

```
Contingency Table:
 [[ 186  337 1229]
 [1036  991 1753]
 [1146  578  604]]
Purity score of the clustering: [0.70148402 0.46375661 0.49226804]
```

[22]:
```
clustering_result2 = {
    'DBSCAN': dbscan.labels_,
    'K-means++': k_means.labels_
}
for name, y_pred in clustering_result2.items():
    print(f"\n{name}:")
    ari = adjusted_rand_score(ground_truth, y_pred)
    nmi = normalized_mutual_info_score(ground_truth, y_pred)
    print(f"ARI: {ari:.4f}, NMI: {nmi:.4f}")
```

```
DBSCAN:
ARI: -0.0021, NMI: 0.0017

K-means++:
ARI: 0.0454, NMI: 0.0603
```

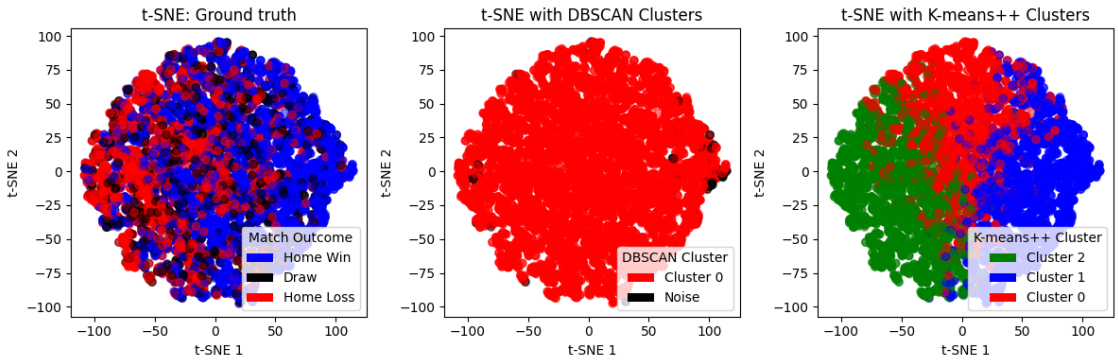
Looking at the contingency tables, K-Means++ performes better than DBSCAN here, with one of its clusters having a 70% ground truth purity.

However both DBSCAN and K-Means++ struggle to find meaningful clusters in the data. The removal of red and yellow card features likely reduced the separability of the data making clustering more difficult. The results suggest that the features used may not be sufficient to distinguish between match outcomes. The ARI and NMI scores are still very low, indicating that the clusters do not align very well with the ground truth.

1.10 T-SNE without disciplinary actions

[23]:
```
# Fit + transform
tsne = TSNE(2, perplexity=10)
embedding = tsne.fit_transform(modified_stats.values)
```

[24]:
```
# --- Updated for single cluster DBSCAN ---
color_map3 = {2: 'green', 1: 'blue', 0: 'red', -1: 'black'}
colors3 = [color_map3[int(val)] for val in labels]
legend_3 = {0: 'Cluster 0', -1: 'Noise'}

# --- Create figure ---
fig, ax = plt.subplots(1, 3, figsize=(12, 4))
ax = ax.flatten()

# --- Subplot 2: Match outcome ---
ax[0].scatter(embedding[:, 0], embedding[:, 1], color=colors2, alpha=0.5)
ax[0].set_xlabel("t-SNE 1")
ax[0].set_ylabel("t-SNE 2")
ax[0].set_title("t-SNE: Ground truth")
legend_elements2 = [Patch(facecolor=color_map2[key], label=legend_2[key]) for␣
 ↪key in legend_2]
ax[0].legend(handles=legend_elements2, title="Match Outcome")

# --- Subplot 3: DBSCAN clusters ---
ax[1].scatter(embedding[:, 0], embedding[:, 1], color=colors3, alpha=0.5)
ax[1].set_xlabel("t-SNE 1")
ax[1].set_ylabel("t-SNE 2")
ax[1].set_title("t-SNE with DBSCAN Clusters")
legend_elements3 = [Patch(facecolor=color_map3[key], label=legend_3[key]) for␣
 ↪key in legend_3]
ax[1].legend(handles=legend_elements3, title="DBSCAN Cluster")

# --- Subplot 4: K-means++ clusters ---
ax[2].scatter(embedding[:, 0], embedding[:, 1], color=colors4, alpha=0.5)
ax[2].set_xlabel("t-SNE 1")
ax[2].set_ylabel("t-SNE 2")
ax[2].set_title("t-SNE with K-means++ Clusters")
legend_elements4 = [Patch(facecolor=color_map4[key], label=legend_4[key]) for␣
 ↪key in legend_4]
ax[2].legend(handles=legend_elements4, title="K-means++ Cluster")

plt.tight_layout()
plt.show()
```



1.10.1 Clustering Results Discussion (Final synthesis and reflections)

The clustering results shown in the t-SNE projections and the clustering evaluations indicate that the selected features do not form clearly separable groups that correspond to match outcomes. In the ground truth visualization, the three outcome classes (home win, draw, and home loss) are heavily intermixed across the embedding space, suggesting that the underlying feature space does not naturally partition according to these labels.

The *DBSCAN* clustering without disciplinary sanctions result further highlights this issue. Almost all samples are assigned to a single cluster, with only a small number identified as noise points. This indicates that the density-based method is unable to find meaningful separations in the data using the current feature representation.

The *K-means++* clustering approach forces the data into three clusters, but these clusters do not align well very well with the ground truth. Instead, the clusters appear to divide the space with brute force.

Overall, these results suggest that the chosen in-game features most likely do not contain a strong natural clustering structure that corresponds to match outcomes. This makes unsupervised clustering a poor approach for directly predicting the result of a match using this feature set.

However, the t-SNE visualization does hint at the presence of a gradual gradient across the embedding space, particularly along the horizontal axis. Rather than forming distinct groups, the data seems to transition smoothly from one region to another. This indicates that match outcomes are influenced by continuous relationships between features rather than discrete groupings.

1.11 Graph-Based analysis (part II)

The objective of this graph-based analysis is to examine whether graph mining techniques can determine team strength from in-game statistics across multiple seasons and identify any anomalous patterns at the team or season level. Each match is represented as an edge, weighted by a combination of performance statistics such as shots, corners, fouls, and cards. By applying PageRank to the resulting graph, we aim to derive a ranking that reflects team strength.

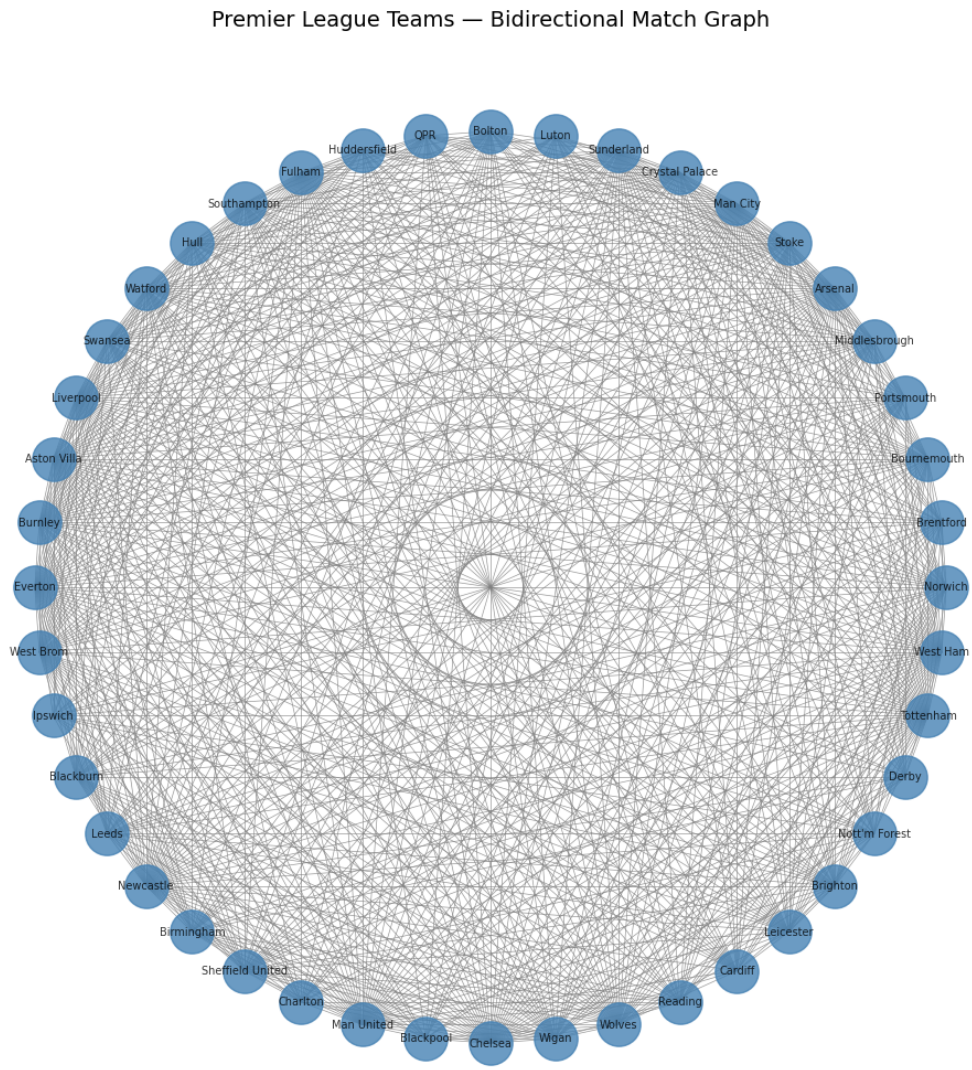
As ground truth, we use a normalised league table, allowing us to evaluate how well the graph-based ranking aligns with the true competitive standing of each team. The closer the PageRank ranking is to the normalised points table, the more effectively the graph captures real team strength.

1.11.1 Graph based representation of data

[25]:
```
data = final_df.copy()

data = pd.DataFrame({'SD': data['HS'] - data['AS'],
                     'SOTD': data['HST'] - data['AST'],
                     'F_diff': data['HF'] - data['AF'],
                     'C_diff': data['HC'] - data['AC'],
                     'YC_diff': data['HY'] - data['AY'],
                     'AvgOdds_diff': data['AvgH'] - data['AvgA'],
                     'RC_diff': data['HR'] - data['AR'],
                     'FTG_diff': data['FTHG'] - data['FTAG']
                     })

# --- Normalize the new features with average and standard deviation (Z-score␣
 ↪normalization) ---
data = (data - data.mean()) / data.std()
data['HomeTeam'] = final_df['HomeTeam']
data['AwayTeam'] = final_df['AwayTeam']
data['Season'] = final_df['Season']
data['FTR'] = final_df['FTR']

base_G = nx.Graph()

# Add all unique teams as nodes
teams = set(data['HomeTeam']).union(set(data['AwayTeam']))
base_G.add_nodes_from(teams)

# Add edges (one per unique matchup)
edges = set()
for _, row in data.iterrows():
    edge = tuple(sorted([row['HomeTeam'], row['AwayTeam']]))
    edges.add(edge)

base_G.add_edges_from(edges)

# Plot
plt.figure(figsize=(10, 10))
pos = nx.circular_layout(base_G)

nx.draw(
    base_G, pos,
    with_labels=True,
    node_color='steelblue',
    node_size=800,
    font_size=7,
    font_color='k',
```

```
    edge_color='gray',
    alpha=0.8,
    width=0.5
)

plt.title("Premier League Teams - Bidirectional Match Graph", fontsize=14)
plt.tight_layout()
plt.show()

print(f"Nodes (teams): {base_G.number_of_nodes()}")
print(f"Edges (unique matchups): {base_G.number_of_edges()}")
```

```
/var/folders/47/5lpcs0y55x31hsb3wzmp02bh0000gn/T/ipykernel_26628/454744202.py:51
: UserWarning: This figure includes Axes that are not compatible with
tight_layout, so results might be incorrect.
  plt.tight_layout()
```




```
Nodes (teams): 44
Edges (unique matchups): 722
```

[26]:
```python
#---- Computing table of points----
def compute_points_table(df):
    """
    Computes the points table from a dataframe
    Parameters:
        df: DataFrame containing at least the columns 'HomeTeam', 'AwayTeam',␣
↪and 'FTR' (Full Time Result)
    Returns:
```

```
        sorted_points: list of tuples (team, points) sorted by points in␣
↪descending order
    """
    # Get all unique teams
    teams = pd.unique(df[['HomeTeam', 'AwayTeam']].values.ravel())

    # Initialize points dictionary
    points = {team: 0 for team in teams}

    # Iterate over matches
    for _, row in df.iterrows():
        if row['FTR'] == 'H':
            points[row['HomeTeam']] += 3
        elif row['FTR'] == 'A':
            points[row['AwayTeam']] += 3
        elif row['FTR'] == 'D':
            points[row['HomeTeam']] += 1
            points[row['AwayTeam']] += 1

    # Convert to sorted list (like standings)
    sorted_points = sorted(points.items(), key=lambda x: x[1], reverse=True)

    return sorted_points


#---- Normalizing table of points----
def normalize_points(points_table: list[tuple[str, float]], df: pd.DataFrame)␣
↪-> list[tuple[str, float]]:
    """
    The normalize_points function computes two different normalized rankings of␣
↪teams based on their accumulated points.
    See documentation in markdown below.

    Parameters:
        points_table: list of tuples (team, points), representing total points␣
↪earned by each team
        df: DataFrame containing match data with at least the columns␣
↪'HomeTeam' and 'AwayTeam'

    Returns:
        total_points_normal: list of tuples (team, normalized_score) where each␣
↪team's score is divided
                                by the total points across all teams, sorted in␣
↪descending order
```

```
        points_pr_match: list of tuples (team, normalized_score) where each␣
↪team's points are first
                        divided by the number of games played, then normalized␣
↪across all teams
                        to sum to 1, sorted in descending order
    """
    # Points/match normalization
    games_played = (
        df.groupby('HomeTeam').size()
        .add(df.groupby('AwayTeam').size(), fill_value=0)
        .astype(int)
    )
    points_pr_match = [
        (team, points / (games_played[team]))
        for team, points in points_table
    ]
    normalisation_factor = sum(points for _, points in points_pr_match)
    points_pr_match = [(team, score / normalisation_factor) for team, score in␣
↪points_pr_match]

    # Total points normalization
    total_points_all_teams = sum(points for _, points in points_table)


    total_points_normal = [(team, points / total_points_all_teams) for team,␣
↪points in points_table]

    return sorted(total_points_normal, key=lambda x: x[1], reverse=True),␣
↪sorted(points_pr_match, key=lambda x: x[1], reverse=True)

norm_pts_1, norm_pts_2 = normalize_points(compute_points_table(data), data)
assert abs(sum(score for _, score in norm_pts_1) - 1.0) < 1e-10
assert abs(sum(score for _, score in norm_pts_2) - 1.0) < 1e-10
```

1.11.2 Baseline method

We start by calculating the pagerank of an undirected unweighted graph where each unique team is nodes and if two teams have played eachother they have an edge. The graph is plotted above.

In the following we have investigated two ways of calculating the loss and two ways of normalizing the points table: #### Loss functions **Rank Loss** Compares the ordering produced by PageRank against the league table ordering. For each team, it computes the squared difference between its rank position in the PageRank output and its rank position in the points table, then averages across all teams. A perfect score of 0 means PageRank reproduces the league standings in exactly the correct order.

Score Loss Compares the values produced by PageRank directly against the normalised points. For each team, it computes the squared difference between its raw PageRank score and its normalised

points value, then averages across all teams. This penalises not just ordering mistakes but also how far apart the two scores are numerically.

Ground truth normalisation methods **Total points Normalisation (norm_pts_2)** Each team's points are divided by the total points distributed across all teams in the season, giving each team a share of the overall points pool. A team's score therefore represents its fraction of all points won, and the scores sum to 1 across the league.

points pr. match Normalisation (norm_pts_1) Each team's points are first divided by the number of played matches, and later normalized giving a win rate bounded between 0 and 1 and summing to 1 across all teams. This accounts for teams that played different numbers of matches.

[27]:
```
def print_ranking_comparison(
    pagerank_ranking: list[tuple[str, float]],
    points_rankings: dict[str, list[tuple[str, float]]],
    losses: dict[str, float] | None = None,
) -> None:
    """
    Side-by-side comparison of PageRank ranking vs. an arbitrary number of␣
↪reference rankings.

    Parameters:
        pagerank_ranking: list of (team, score) sorted by PageRank descending.
        points_rankings: dict of {label: list of (team, score)}, each sorted␣
↪descending.
        losses: optional dict of {label: loss_value} to print below the table.
    """
    col_width = 25
    score_width = 10

    header = f"{'Rank':<6} {'PageRank Team':<{col_width}} {'PR Score':<12}"
    for label in points_rankings:
        header += f" {label:<{col_width}} {'Score':<{score_width}}"
    print(header)
    print("-" * (6 + 12 + col_width + (col_width + score_width + 2) *␣
↪len(points_rankings)))

    for i, (pr_team, pr_score) in enumerate(pagerank_ranking, 1):
        row = f"{i:<6} {pr_team:<{col_width}} {pr_score:<12.4f}"
        for ranking in points_rankings.values():
            if i <= len(ranking):
                pt_team, pt_score = ranking[i - 1]
                row += f" {pt_team:<{col_width}} {pt_score:<{score_width}.4f}"
        print(row)

    print("-" * (6 + 12 + col_width + (col_width + score_width + 2) *␣
↪len(points_rankings)))
```

```
    if losses:
        for loss_name, loss_value in losses.items():
            print(f"Loss {loss_name}: {loss_value:.6f}")

def rank_loss(pagerank_scores, norm_pts):
    """
    Loss function that penalises rank disagreement between PageRank and␣
↪normalised points.
    For each team, the penalty is the squared difference in rank position␣
↪between the two tables.
    E.g. if Liverpool is rank 1 in PageRank and rank 2 in the points table, the␣
↪penalty is (1-2)^2 = 1.

    Parameters:
        pagerank_scores: list of tuples (team, pagerank_score) sorted by score␣
↪descending.
        norm_pts: list of tuples (team, normalized_points) sorted by score␣
↪descending.

    Returns:
        float: mean squared rank difference across all teams present in both␣
↪tables.
    """
    pr_ranks  = {team: rank for rank, (team, _) in enumerate(pagerank_scores,␣
↪1)}
    pts_ranks = {team: rank for rank, (team, _) in enumerate(norm_pts, 1)}

    teams = [t for t in pr_ranks if t in pts_ranks]
    return sum((pr_ranks[t] - pts_ranks[t]) ** 2 for t in teams) / len(teams)

def score_loss(pagerank_scores, norm_pts):
    """
    Loss function to evaluate how well the pagerank scores align with the␣
↪normalised points.
    Parameters:
        pagerank_scores: list of tuples (team, pagerank_score)
        norm_pts: list of tuples (team, normalized_points)
    Returns:
        loss_value: float, the mean squared error between pagerank scores and␣
↪normalized points for teams
    """
    pr_dict  = dict(pagerank_scores)
    pts_dict = dict(norm_pts)
    teams    = [t for t in pr_dict if t in pts_dict]
    return sum((pr_dict[t] - pts_dict[t]) ** 2 for t in teams) / len(teams)
```

```python
pR = nx.pagerank(base_G, alpha=0.85)
sorted_pagerank = dict(sorted(pR.items(), key=lambda x: x[1], reverse=True))
base_loss_1 = score_loss(list(sorted_pagerank.items()), norm_pts_1)
base_loss_2 = score_loss(list(sorted_pagerank.items()), norm_pts_2)
base_table_loss_1, base_table_loss_2 = rank_loss(list(sorted_pagerank.items()),␣
↪norm_pts_1), rank_loss(list(sorted_pagerank.items()), norm_pts_2)
losses = {
    "(score) Total points normalized": base_loss_1,
    "(score) Points/match normalized": base_loss_2,
    "(rank) total points": base_table_loss_1,
    "(rank) points/match": base_table_loss_2
}
ground_truths = {
    "Total points normalized": norm_pts_1,
    "Points/match normalized": norm_pts_2
}

print_ranking_comparison(list(sorted_pagerank.items()), ground_truths, losses)
```

```
Rank   PageRank Team             PR Score     Total points normalized   Score
Points/match normalized   Score
--------------------------------------------------------------------------------
-------------------------------------
1      Arsenal                   0.0290       Man City                  0.0723
Man City                  0.0387
2      Man City                  0.0290       Liverpool                 0.0708
Liverpool                 0.0380
3      Fulham                    0.0290       Man United                0.0705
Man United                0.0378
4      Liverpool                 0.0290       Chelsea                   0.0701
Chelsea                   0.0376
5      Everton                   0.0290       Arsenal                   0.0692
Arsenal                   0.0371
6      Newcastle                 0.0290       Tottenham                 0.0608
Tottenham                 0.0326
7      Man United                0.0290       Everton                   0.0515
Everton                   0.0276
8      Chelsea                   0.0290       West Ham                  0.0423
Brentford                 0.0262
9      Tottenham                 0.0290       Newcastle                 0.0422
Leicester                 0.0258
10     West Ham                  0.0290       Aston Villa               0.0405
Aston Villa               0.0254
11     Aston Villa               0.0283       Fulham                    0.0295
Newcastle                 0.0251
12     Wolves                    0.0265       Crystal Palace            0.0269
Blackburn                 0.0247
13     Sunderland                0.0264       Southampton               0.0237
```

				Brighton	0.0242
14	Burnley	0.0258		Leicester	0.0233
				West Ham	0.0238
15	West Brom	0.0258		Stoke	0.0211
				Crystal Palace	0.0235
16	Watford	0.0245		Wolves	0.0209
				Stoke	0.0234
17	Southampton	0.0240		Sunderland	0.0203
				Nott'm Forest	0.0230
18	Stoke	0.0240		West Brom	0.0198
				Bournemouth	0.0230
19	Crystal Palace	0.0234		Brighton	0.0190
				Swansea	0.0228
20	Bournemouth	0.0228		Bournemouth	0.0180
				Bolton	0.0227
21	Leicester	0.0228		Burnley	0.0169
				Portsmouth	0.0223
22	Norwich	0.0227		Blackburn	0.0156
				Fulham	0.0223
23	Wigan	0.0222		Wigan	0.0153
				Leeds	0.0219
24	Sheffield United	0.0222		Swansea	0.0144
				Southampton	0.0219
25	Swansea	0.0221		Bolton	0.0143
				Wolves	0.0217
26	Bolton	0.0216		Watford	0.0120
				Wigan	0.0212
27	Blackburn	0.0216		Brentford	0.0111
				Charlton	0.0207
28	Brighton	0.0216		Portsmouth	0.0101
				Reading	0.0203
29	Middlesbrough	0.0216		Norwich	0.0093
				Birmingham	0.0202
30	Hull	0.0215		Middlesbrough	0.0089
				West Brom	0.0200
31	Reading	0.0198		Hull	0.0079
				Blackpool	0.0200
32	Portsmouth	0.0192		Nott'm Forest	0.0077
				Middlesbrough	0.0197
33	Birmingham	0.0187		Birmingham	0.0073
				Burnley	0.0194
34	Brentford	0.0186		Leeds	0.0073
				Sunderland	0.0193
35	Cardiff	0.0185		Sheffield United	0.0060
				Watford	0.0191
36	QPR	0.0185		Reading	0.0055
				Hull	0.0175
37	Leeds	0.0179		QPR	0.0042

```
Norwich                    0.0171
38     Nott'm Forest              0.0175       Charlton                  0.0037
Sheffield United           0.0168
39     Charlton                   0.0162       Cardiff                   0.0030
Cardiff                    0.0164
40     Huddersfield               0.0162       Huddersfield              0.0024
QPR                        0.0157
41     Derby                      0.0145       Blackpool                 0.0018
Huddersfield               0.0136
42     Ipswich                    0.0144       Luton                     0.0012
Luton                      0.0133
43     Luton                      0.0144       Ipswich                   0.0010
Ipswich                    0.0113
44     Blackpool                  0.0144       Derby                     0.0005
Derby                      0.0056
--------------------------------------------------------------------------------
-------------------------------------
Loss (score) Total points normalized: 0.000324
Loss (score) Points/match normalized: 0.000024
Loss (rank) total points: 20.681818
Loss (rank) points/match: 114.909091
```

Note that Blackpool was playing in premier league for one season and was relegated with 39 points. It is quite unusual to be relegated with such a high number of points. This is evident when examining the points/match normalized score (ranked 31/44). In the PageRank they are ranked 44/44 and in total points normalized they are ranked 41/44.

1.12 More sophisticated graph-based method

To capture more nuanced relationships between teams, we construct a weighted directed graph where edges represent matches and weights are derived from in-game performance statistics. For each match, we calculate a composite score based on features such as shot difference, corner difference, foul difference, and card difference. All matches between two teams are summed and represented by one edge. This score is used to weight the directed edge from the losing team to the winning team, reflecting the relative strength of the two teams.

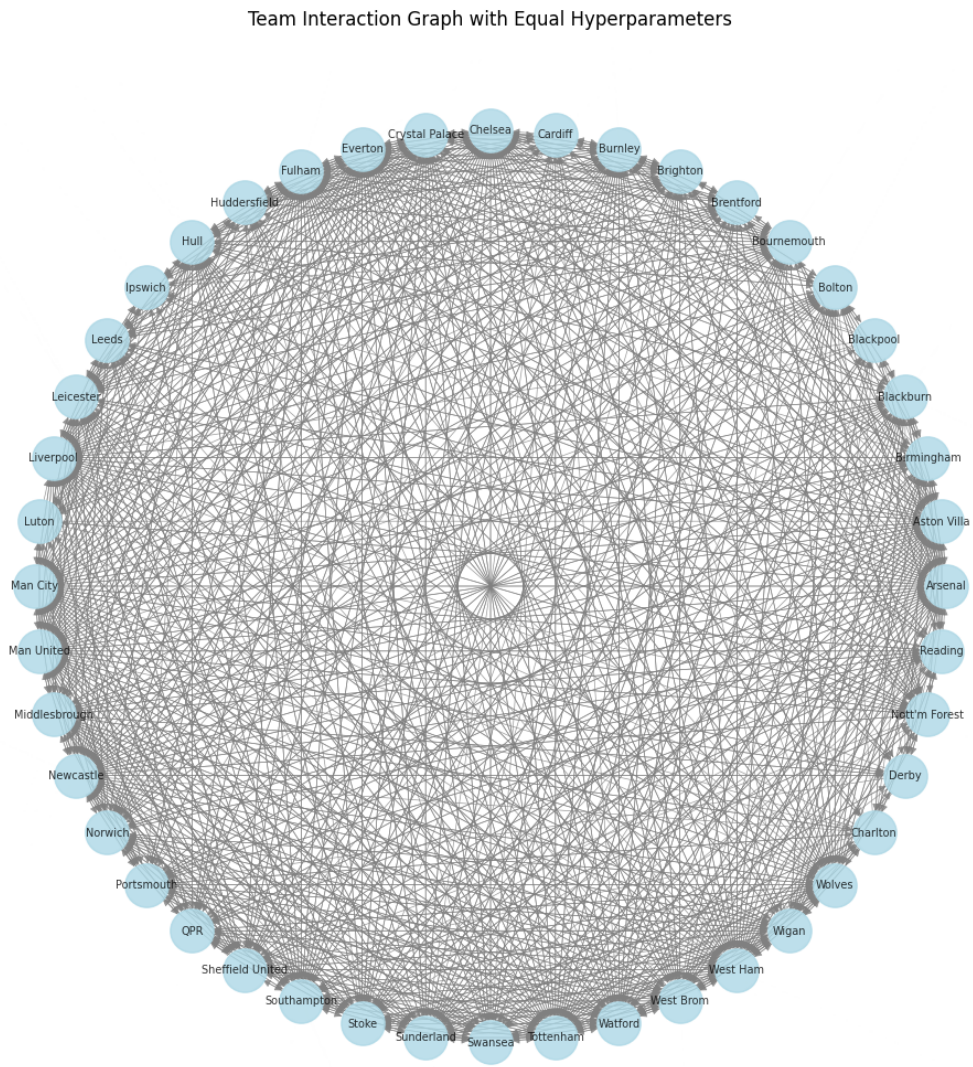
[28]:
```
#   Data structure

class DirectedEdge:
    """A weighted directed edge from `source` to `target`."""
    def __init__(self, source: str, target: str, weight: float):
        self.source = source
        self.target = target
        self.weight = weight

#   Edge construction

def match_stats(row) -> np.ndarray:
```

```
    """
    Extracts per-match differential statistics from a DataFrame row.
    Positive values favour the home team; the vector is L2-normalised.

    Parameters:
        row (pandas.Series): A row from the match DataFrame containing the
            following columns: FTHG, FTAG, HS, AS, HST, AST, HF, AF,
            HC, AC, HY, AY, HR, AR, AvgH, AvgA.

    Returns:
        np.ndarray: A normalised array of 8 differential statistics in the
            order [goals, shots, shots-on-target, fouls, corners,
            yellow-cards, red-cards, odds]. Returns the unnormalised array
            if the L2 norm is zero (e.g. all stats are equal).
    """
    stats = np.array([
        data['FTG_diff'][row.name],   # Goal difference
        data['SD'][row.name],      # Shot difference
        data['SOTD'][row.name],     # Shots on target difference
        data['F_diff'][row.name],      # Foul difference (away - home; fewer␣
↪fouls is better)
        data['C_diff'][row.name],      # Corner difference
        data['YC_diff'][row.name],      # Yellow card difference (away - home;␣
↪fewer cards is better)
        data['RC_diff'][row.name],      # Red card difference   (away - home;␣
↪fewer cards is better)
        data['AvgOdds_diff'][row.name],    # Betting odds difference
    ])
    return stats

def match_row_to_edge(row, hyperparams) -> DirectedEdge | None:
    """
    Converts a match row into a directed edge pointing from loser → winner.
    Returns None for draws (zero weighted score).

    Parameters:
        row (pandas.Series): A row from the DataFrame
        hyperparams (np.ndarray): The hyperparameters for the weighted score␣
↪calculation
    Returns:
        DirectedEdge: A directed edge from the losing team to the winning team␣
↪with weight equal to
            the absolute value of the weighted score, or None if it's a draw.
    """
    stats = match_stats(row)
    weighted_score = np.dot(stats, hyperparams)
```

```
    if weighted_score > 0:
        return DirectedEdge(row['AwayTeam'], row['HomeTeam'],  weighted_score)
    elif weighted_score < 0:
        return DirectedEdge(row['HomeTeam'], row['AwayTeam'], - weighted_score)␣
↪# minus sign to ensure weight is positive
    return None

def compute_edges(data: pd.DataFrame, hyperparams) -> list[DirectedEdge]:
    """
    Applies `match_row_to_edge` to every row and drops draws.

    Parameters:
        data (pd.DataFrame): The DataFrame containing match data.
        hyperparams (np.ndarray): The hyperparameters for the weighted score␣
↪calculation.
    Returns:
       list[DirectedEdge]: A list of directed edges representing the matches,␣
↪excluding draws.

    """

    edges = data.apply(lambda row: match_row_to_edge(row, hyperparams), axis=1)
    return [e for e in edges if e is not None]

#   Graph construction

def edges_to_dataframe(edges: list[DirectedEdge]) -> pd.DataFrame:
    """
    Converts a list of DirectedEdge objects into an aggregated edge DataFrame.
    Multiple edges between the same pair of teams are summed.

    Parameters:
        edges (list[DirectedEdge]): A list of directed edges to convert.
    Returns:
        pd.DataFrame: A DataFrame with columns ['source', 'target', 'weight']␣
↪where weights are summed for duplicate edges.
    """
    df = pd.DataFrame({
        'source': [e.source for e in edges],
        'target': [e.target for e in edges],
        'weight': [e.weight for e in edges],
    })
    return df.groupby(['source', 'target'], as_index=False)['weight'].sum()


def build_graph(edges: list[DirectedEdge]) -> nx.DiGraph:
    """
```

```
    Builds a weighted directed graph from a list of DirectedEdge objects.

    Parameters:
        edges (list[DirectedEdge]): A list of directed edges to build the graph␣
↪from.
    Returns:
        nx.DiGraph: A directed graph where nodes are teams and edges represent␣
↪weighted interactions
    """
    edge_df = edges_to_dataframe(edges)
    return nx.from_pandas_edgelist(
        edge_df,
        source='source',
        target='target',
        edge_attr='weight',
        create_using=nx.DiGraph()
    )

# - Graph visualization
def plot_graph(G: nx.DiGraph, title: str = 'Team Interaction Graph') -> None:
    """
    Plots a directed graph using a spring layout.

    Parameters:
        G (nx.DiGraph): The directed graph to plot.
        title (str): The title for the plot.
    """
    plt.figure(figsize=(10, 10))
    pos = nx.circular_layout(G)
    nx.draw(G, pos, with_labels=True, node_size=800,
            node_color='lightblue', edge_color='gray', font_size=7, alpha=0.8,␣
↪width=0.5, arrowsize=10)
    plt.title(title)
    plt.show()
```

[29]:
```
# Plotting a directed graph with all ones as hyperparameters
plot_graph(build_graph(compute_edges(data, np.ones(8))), title="Team␣
↪Interaction Graph with Equal Hyperparameters")
```



[30]:
```python
#   Transition matrix and PageRank
def build_transition_matrix(G: nx.DiGraph) -> np.ndarray:
    """
    Constructs the column-stochastic transition matrix from a directed graph.
    M = A^T · Δ ¹  (08-LinkAnalysis slide 19)

    Parameters:
        G (nx.DiGraph): The directed graph representing team interactions.
    Returns:
        np.ndarray: The transition matrix
    """
    nodes = sorted(G.nodes())
```

```python
    A     = nx.to_numpy_array(G, weight='weight', nodelist=nodes)
    Delta = np.diag(A.sum(axis=1))
    return A.T @ np.linalg.inv(Delta)


def power_iteration(
    M: np.ndarray,
    max_iter: int = 1000,
    alpha: float  = 0.85,
    tol: float    = 1e-6,
) -> np.ndarray:
    """
    Estimates the PageRank vector via power iteration with teleportation.
    (08-LinkAnalysis slide 22)

    Parameters:
        M (np.ndarray): The column-stochastic transition matrix.
        max_iter (int): Maximum number of iterations to perform.
        alpha (float): Teleportation factor (probability of following a link vs.
 ↪ teleporting).
        tol (float): Convergence tolerance for the norm of rank changes.
    Returns:
        np.ndarray: A probability vector (sums to 1) over all nodes.
    """
    n   = M.shape[0]
    E   = np.ones((n, n)) / n
    M_  = alpha * M + (1 - alpha) * E

    rank = np.ones(n) / n # Initial uniform distribution
    for _ in range(max_iter):
        new_rank = M_ @ rank
        if np.linalg.norm(new_rank - rank, 1) < tol:
            break
        rank = new_rank / new_rank.sum()

    return rank / rank.sum()


def compute_pagerank(G: nx.DiGraph, M: np.ndarray) -> list[tuple[str, float]]:
    """
    Computes the PageRank scores for each node in the graph.

    Parameters:
        G (nx.DiGraph): The directed graph representing team interactions.
        M (np.ndarray): The transition matrix derived from G.
    Returns:
```

```
        list[tuple[str, float]]: A list of (team, pagerank_score) tuples,␣
↪sorted by score descending.

    """
    rank_vector = power_iteration(M)
    scores      = dict(zip(sorted(G.nodes()), rank_vector))
    return sorted(scores.items(), key=lambda x: x[1], reverse=True)


#   Full pipeline

def run_pagerank_pipeline(
    data: pd.DataFrame,
    hyperparams,
    plot: bool = False,
) -> tuple[nx.DiGraph, np.ndarray, list[tuple[str, float]]]:
    """
    Runs the full pipeline: data + hyperparams → edges → graph → transition␣
↪matrix → PageRank ranking.

    Parameters:
        data (pd.DataFrame): The match data.
        hyperparams: The hyperparameters for edge weighting.
        plot (bool): Whether to plot the graph.

    Returns:
        G (nx.DiGraph): The constructed directed graph.
        M (np.ndarray): The transition matrix derived from G.
        sorted_teams (list[tuple[str, float]]): Teams sorted by PageRank score␣
↪(descending).
    """
    edges = compute_edges(data, hyperparams)
    G = build_graph(edges)
    if plot:
        plot_graph(G)
    M = build_transition_matrix(G)
    sorted_teams = compute_pagerank(G, M)
    return G, M, sorted_teams

#   Evaluation & search

def evaluate(data: pd.DataFrame, hyperparams, norm_pts, loss: callable) ->␣
↪float:
    """
    MSE between PageRank scores and normalised league points.

    Parameters:
```

```
        data (pd.DataFrame): The match data.
        hyperparams: The hyperparameters for edge weighting.
        norm_pts: List of (team, normalized_points) to compare against.
        loss: The loss function to evaluate the alignment between PageRank␣
↪scores and normalized points.
    Returns:
        float: The computed loss value.
    """
    _, _, pagerank_scores = run_pagerank_pipeline(data, hyperparams)

    return loss(pagerank_scores, norm_pts)


def random_search(
    data: pd.DataFrame,
    norm_pts,
    n_trials:  int  = 500,
    data_leak: bool = False,
    loss: callable = score_loss
) -> tuple[np.ndarray, float]:
    """
    Randomly samples hyperparameter vectors and keeps the one with lowest MSE.
    Set data_leak=False to zero out goal-difference and odds weights.

    Parameters:
        data (pd.DataFrame): The match data.
        norm_pts: List of (team, normalized_points) to compare against.
        n_trials (int): Number of random hyperparameter vectors to sample.
        data_leak (bool): If False, zero out goal-difference and odds weights␣
↪to prevent data leakage.
        loss (callable): The loss function to evaluate the alignment between␣
↪PageRank
    Returns:
        tuple[np.ndarray, float]: The best hyperparameters found and their␣
↪corresponding loss value.

    """
    best_loss, best_params = float('inf'), np.ones(8)

    for _ in tqdm(range(n_trials), desc="Random Search"):
        params = np.random.uniform(0, 5, size=8)
        if not data_leak:
            params[0] = 0   # Zero out goal-difference weight
            params[7] = 0   # Zero out betting-odds weight

        ls = evaluate(data, params, norm_pts, loss)
        if ls < best_loss:
```

```
            best_loss, best_params = ls, params.copy()

    return best_params, best_loss
```

[31]:
```
# Supervised search for hyperparameters with no data leak (zeroing out␣
↪goal-difference and odds weights)
train_data = final_df.copy()[final_df['Season'] == '20252026premierleague']

random_search_params_1, random_search_loss_1 = random_search(train_data,␣
↪norm_pts_1,
                                                              n_trials=1000,␣
↪data_leak=False, loss=rank_loss)
random_search_params_2, random_search_loss_2 = random_search(train_data,␣
↪norm_pts_2,
                                                              n_trials=1000,␣
↪data_leak=False, loss=rank_loss)
random_search_params_3, random_search_loss_3 = random_search(train_data,␣
↪norm_pts_1,
                                                              n_trials=1000,␣
↪data_leak=False, loss=score_loss)
random_search_params_4, random_search_loss_4 = random_search(train_data,␣
↪norm_pts_2,
                                                              n_trials=1000,␣
↪data_leak=False, loss=score_loss)
```

```
Random Search: 100%|      | 1000/1000 [00:06<00:00, 163.15it/s]
Random Search: 100%|      | 1000/1000 [00:06<00:00, 166.49it/s]
Random Search: 100%|      | 1000/1000 [00:05<00:00, 171.04it/s]
Random Search: 100%|      | 1000/1000 [00:05<00:00, 173.23it/s]
```

[32]:
```
best_losses = {
    "Score loss Total points normalized": evaluate(data,␣
↪random_search_params_3, norm_pts_1, loss=score_loss),
    "Score loss Points/match normalized": evaluate(data,␣
↪random_search_params_4, norm_pts_2, loss=score_loss),
    "Rank loss total points": evaluate(data, random_search_params_1,␣
↪norm_pts_1, loss=rank_loss),
    "Rank loss points/match": evaluate(data, random_search_params_2,␣
↪norm_pts_2, loss=rank_loss)
}
best_hyperparams = {
    "Score loss Total points normalized": random_search_params_3,
    "Score loss Points/match normalized": random_search_params_4,
    "Rank loss total points": random_search_params_1,
    "Rank loss points/match": random_search_params_2
}
```

```python
# Printing the best losses for each evaluation metric
for i in best_losses:
    print(f"{i}: {best_losses[i]:.6f}")

# Printing the best hyperparameters for each evaluation metric
print('-' * 50)
for i in best_hyperparams:
    print(f"Best hyperparameters for {i}: {best_hyperparams[i]}")
```

```
Score loss Total points normalized: 0.000062
Score loss Points/match normalized: 0.000282
Rank loss total points: 11.363636
Rank loss points/match: 54.772727
--------------------------------------------------
Best hyperparameters for Score loss Total points normalized: [0.
3.50064781 4.98410725 0.16821911 0.32616331 0.38975035
 3.71489567 0.        ]
Best hyperparameters for Score loss Points/match normalized: [0.
4.73253661 0.71685156 0.00981009 0.38439107 2.31354148
 3.53466015 0.        ]
Best hyperparameters for Rank loss total points: [0.         3.46974795
1.1479908  0.5877565  0.82642245 0.00996068
 3.59369809 0.        ]
Best hyperparameters for Rank loss points/match: [0.         3.03415714
4.3036428  0.07707479 0.53617901 0.46288034
 3.56302342 0.        ]
```

These obtained weights are interpreted as a measure of dominance in the matchups.

The most effective loss function proved to be the score loss. This metric directly captures how well the PageRank ordering reflects the actual competitive standings, making it an intuitive and interpretable measure of model quality. It also became apparent that using points per game normalization in combination with score loss, led to a significantly lower loss for the baseline method. However, this is a result of the baseline method's structure rather than a genuine signal of performance.

The baseline model is constructed on a bidirectional graph, which creates a systematic bias for teams with limited participation. Consider Blackpool as an example: they appear in only one of the analyzed Premier League seasons, after which they were relegated, losing a significant proportion of their matches which is assumingly evident in the statistics. In a bidirectional graph, a random walk can reach Blackpool from all 19 opposing teams in the league, regardless of match statistics. In a directed graph, however, only the teams that Blackpool actually dominated will have edges pointing toward Blackpool, meaning the random walk can arrive at Blackpool from only a fraction of those 19 teams. This results in a lower PageRank score, which will increase the score loss measured against points pr. match ground truth as this ground truth measure has a small variance.

The score loss is onwards selected as the primary evaluation metric. This choice is based on the fact that it provides a continuous measure of alignment between PageRank scores and the league table than the discrete rank differences of the rank loss.

Overall, based on the presented analysis, we decided to use the score loss and the total points normalization, as this seems to fit better with our analysis goal.

1.13 Seasonal Losses:

In the following two cells, a random search for the best set of hyperparameters has been done, trained on all the available seasons. These hyperparameters are used as a ground truth, as they should reflect how football matches in general behave. Afterwards, the score loss is calculated for each season, to check whether some seasons were abnormal. The whole procedure is iterated ten times to see if the random walk is consistent, i.e. that the found abnormal seasons are consistent.

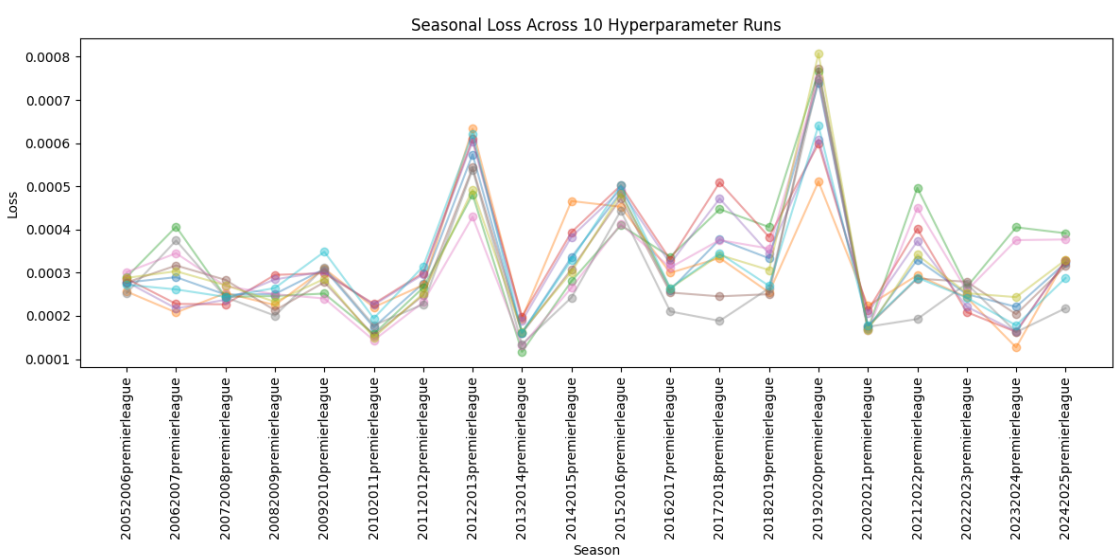
[33]:
```
seasons = ['20052006premierleague',
           '20062007premierleague',
           '20072008premierleague',
           '20082009premierleague',
           '20092010premierleague',
           '20102011premierleague',
           '20112012premierleague',
           '20122013premierleague',
           '20132014premierleague',
           '20142015premierleague',
           '20152016premierleague',
           '20162017premierleague',
           '20172018premierleague',
           '20182019premierleague',
           '20192020premierleague',
           '20202021premierleague',
           '20212022premierleague',
           '20222023premierleague',
           '20232024premierleague',
           '20242025premierleague']

n_runs = 10
all_losses = []   # shape: (n_runs, n_seasons)
all_hyperparams = []

for run in range(n_runs):
    print(f"Run {run+1}/{n_runs}")

    # Step 1: New hyperparameter search each time
    best_hyperparams, best_loss = random_search(
        data,
        norm_pts_1,
        n_trials=1000,
        data_leak=False,
        loss=score_loss
    )
```

```
    all_hyperparams.append(best_hyperparams)

    # Step 2: Compute seasonal losses
    loss = np.zeros(len(seasons))

    for i in range(len(seasons)):
        season = seasons[i]
        season_data = data[data['Season'] == season]

        norm_pts_season = normalize_points(
            compute_points_table(season_data),
            season_data
        )[0]

        loss_season = evaluate(
            season_data,
            best_hyperparams,
            norm_pts_season,
            loss=score_loss
        )

        loss[i] = loss_season

    all_losses.append(loss)

all_losses = np.array(all_losses)  # shape (10, n_seasons)
```

```
Run 1/10
Random Search: 100%|      | 1000/1000 [02:49<00:00,  5.90it/s]
Run 2/10
Random Search: 100%|      | 1000/1000 [02:48<00:00,  5.94it/s]
Run 3/10
Random Search: 100%|      | 1000/1000 [02:46<00:00,  6.00it/s]
Run 4/10
Random Search: 100%|      | 1000/1000 [02:47<00:00,  5.99it/s]
Run 5/10
Random Search: 100%|      | 1000/1000 [02:46<00:00,  6.00it/s]
Run 6/10
Random Search: 100%|      | 1000/1000 [02:48<00:00,  5.94it/s]
Run 7/10
Random Search: 100%|      | 1000/1000 [02:48<00:00,  5.95it/s]
```

```
Run 8/10
Random Search: 100%|      | 1000/1000 [02:47<00:00,  5.96it/s]
Run 9/10
Random Search: 100%|      | 1000/1000 [02:48<00:00,  5.94it/s]
Run 10/10
Random Search: 100%|      | 1000/1000 [02:48<00:00,  5.93it/s]
```

[34]:
```
# --- Plotting the seasonal losses across runs ---

fig, ax = plt.subplots(figsize=(12, 6))

for run in range(n_runs):
    ax.plot(seasons, all_losses[run], marker='o', alpha=0.4)

ax.set_xticks(range(len(seasons)))
ax.set_xticklabels(seasons, rotation=90)
ax.set_xlabel('Season')
ax.set_ylabel('Loss')
ax.set_title('Seasonal Loss Across 10 Hyperparameter Runs')

plt.tight_layout()
plt.show()
```



1.13.1 Evaluation of the seasonal loss plot

The plot shows a consistently higher loss for especially season 2012/2013 and 2019/2020. For both seasons it is hard to determine exactly the reason for this relatively high loss.

However, for season 2019/2020, some unsual things happened during the season: - Corona, meaning the home advantage disappeared. - Liverpool had the fastest title win ever (7 matches remaining).

1.14 Team losses

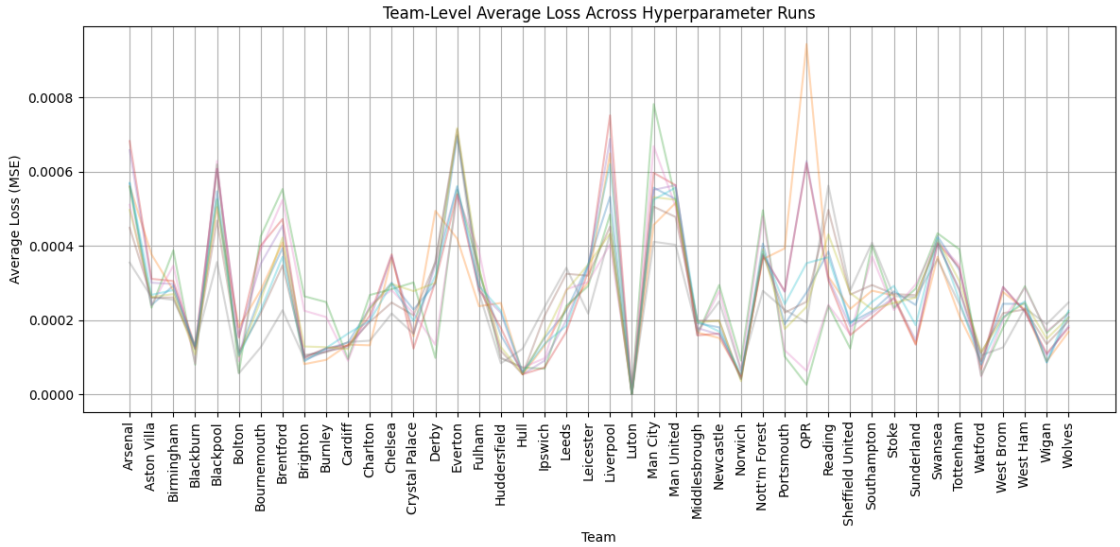
The next two cells uses the earlier found best hyperparameters of all seasons i.e. the list **all_hyperparams**, to calculate the score loss of each team. This is done to reveal if some teams play in a way different from the other teams.

This is done by running through all seasons and calculating each teams score loss in each season. Since the calculated team loss is summed over all the seasons a team has played the Premier League, each team loss is normalized by the number of seasons played by that team.

[35]:
```
all_team_losses = []


# Fix global team universe (important for alignment)
all_teams = set()

# First pass to collect all teams across all seasons
for season in seasons:
    season_data = data[data['Season'] == season]
    norm_pts_season = normalize_points(
        compute_points_table(season_data),
        season_data
    )[0]
    all_teams.update([team for team, _ in norm_pts_season])

reference_teams = np.array(sorted(all_teams))

for hyp in range(len(all_hyperparams)):

    # Step 1: hyperparameters
    best_hyperparams = all_hyperparams[hyp]

    # Step 2: accumulate per-team loss across seasons
    team_loss_sum = {team: 0.0 for team in reference_teams}
    team_season_count = {team: 0 for team in reference_teams}

    for season in seasons:
        season_data = data[data['Season'] == season]

        norm_pts_season = normalize_points(
            compute_points_table(season_data),
            season_data
        )[0]
        norm_pts_dict = dict(norm_pts_season)
```

```
        pagerank = run_pagerank_pipeline(season_data, best_hyperparams)[2]
        pagerank_dict = dict(pagerank)

        for team in norm_pts_dict:
            if team not in pagerank_dict:
                continue

            pr_score = pagerank_dict[team]
            gt_score = norm_pts_dict[team]

            loss = (pr_score - gt_score) ** 2

            team_loss_sum[team] += loss
            team_season_count[team] += 1

    # Step 3: compute average loss per team (aligned vector)
    losses = np.zeros(len(reference_teams))

    for i, team in enumerate(reference_teams):
        if team_season_count[team] == 0:
            losses[i] = np.nan
        else:
            losses[i] = team_loss_sum[team] / team_season_count[team]

    all_team_losses.append(losses)

all_team_losses = np.array(all_team_losses)  # shape: (n_runs, n_teams)
```

[36]:
```
fig, ax = plt.subplots(figsize=(12, 6))

x = np.arange(len(reference_teams))

for losses in all_team_losses:
    ax.plot(x, losses, alpha=0.3)

ax.set_xticks(x)
ax.set_xticklabels(reference_teams, rotation=90)

ax.set_xlabel('Team')
ax.set_ylabel('Average Loss (MSE)')
ax.set_title('Team-Level Average Loss Across Hyperparameter Runs')

plt.tight_layout()
plt.grid()
plt.show()
```



1.14.1 Evaluation of the team loss plot

In the chart (not graph!) we see the teamwise losses across all the seasons. It is a bit hard to decode the outcome of this analysis.

However, some teams do spike out such as Liverpool and Manchester City, which are widely considered the two best teams in premier league, and this is by itself abnormal.

Blackpool also has a spike, this is probably due to Blackpool being a trash team, but probably also due to the fact that the one season they played, they got relegated despite earning 39 points, which is unsual.

The model seems to have a hard time determining the loss of Queens Park Rangers, for some reason their loss is very sensitive to different hyperparameters.

1.14.2 Further investigations could include

Comparing title winning team games, in stead of all games across the season. This could reveal: - The unsual Leicester title win in 2015/2016, or - The 2018/2019 season were was widely considered to be unlucky not winning the title.

1.14.3 How the feedback was from checkpoint 1 was adressed

- A manhattan distance was introduced when deciding the hyperparameters for DBSCAN. However, due to the sensitivity in the number of clusters when changing ϵ, this was quickly disregarded.
- ARI and NMI evaluation of both DBSCAN and K-Means++ was implemented to introduce some quantitative analysis of the perfance of algorithms.